

INFORME DE ESTADO DEL PROYECTO

ProLegends

Explorador local del archivo de leyendas de Dwarf Fortress

Repaso completo de lo construido hasta la versión 1.5.1: qué hace cada fichero, qué decisiones se tomaron y por qué, qué está comprobado, qué sigue abierto y por dónde continúa el trabajo.

Versión revisada 1.5.1

Repositorio HerrKlicova/ProLegends-Dwarf-Fortress · rama `main`

Commits 34 · **Versiones publicadas** 18

Fecha 10 de septiembre de 2026

1 En una página

ProLegends lee los dos ficheros que Dwarf Fortress escribe cuando exportas las leyendas de un mundo — `legends.xml` y `legends_plus.xml`, entre 20 y 60 MB— y los convierte en algo que se puede recorrer: un mapa del mundo dibujado a partir de los datos, fichas de personajes y de sitios, un panel de tu fortaleza, y crónicas narradas por IA cuando tú las pides.

Se arranca con doble clic. No hay servidor en internet, no hay cuenta, no hay nada que instalar más allá de Python. Todo ocurre en tu ordenador y todo está en castellano.

16.355 líneas de código y documentación	54 ficheros de código
29 tablas en la base de datos	31 endpoints de la API interna
155 plantillas de narración de sucesos	557 términos traducidos de DF a castellano

BLOQUE	FICHEROS	LÍNEAS	QUÉ ES
Backend <code>app/</code>	37	9.713	Lectura del XML, base de datos, API interna, narrador, IA
Interfaz <code>web/</code>	11	3.314	HTML, CSS y JavaScript sin compilación
Herramientas <code>tools/</code>	2	1.544	Autocomprobación y generador de mundos falsos
Documentación	4	1.784	README, CHANGELOG, ROADMAP, análisis de LegendsViewer
Total	54	16.355	

LAS CUATRO DEPENDENCIAS

`fastapi` y `uvicorn` para el servidor local, `python-dotenv` para leer tu clave y `anthropic` para las crónicas. Nada más. Ni Node, ni npm, ni ningún paso de compilación: fue una condición desde el principio y se ha respetado entera.

2 El camino recorrido, versión a versión

Dieciocho versiones publicadas, cada una con su entrada en el `CHANGELOG.md`. Lo que sigue no es la lista de cambios —esa ya está escrita— sino *por qué* se hizo cada una.

VERSIÓN QUÉ RESOLVIÓ

1.0	El cimiento: leer el XML sin ahogarse, guardarlo en SQLite y enseñarlo. Aquí se decidió casi todo lo que vino después.
1.1	Rendimiento con mundos grandes, colores de facción derivados de los propios datos y la primera autocomprobación.
1.2	Las crónicas dejan de vivir en la base de datos y pasan a ser ficheros. Son lo único que cuesta dinero: no podían depender de una tabla que se borra.
1.3.0	La aplicación encuentra sola tu instalación de Dwarf Fortress y se trae los exports. Antes había que copiarlos a mano.
1.3.1	Tres cosas que estorbaban al actualizar: la interfaz vieja en caché, la importación lenta y las crónicas atrapadas en la carpeta del programa.
1.3.2	La mudanza a la carpeta personal también mira en las instalaciones anteriores. Se descubrió porque a ti no te encontró nada.
1.4.0	El mapa de pergamino, dibujado con los datos del propio export. El salto visual más grande del proyecto.
1.4.1- 1.4.4	Los ríos. Cuatro intentos. La historia está en el apartado 5, porque es la lección más útil que ha dado el proyecto.
1.5.0	Los sucesos se cuentan en castellano. Un diccionario y un narrador convierten <code>hf died · cause: STRUCK</code> en una frase que se lee.
1.5.1	Tus ocho comentarios sobre la interfaz, contestados. Y el fallo de fondo que escondían: cada mundo escribe los nombres de los sucesos a su manera.

3 Recorrido por el código

Cada fichero, qué hace y por qué está separado de los demás.

El cimiento: leer el XML

FICHERO	QUÉ HACE
<code>parser/xmlstream.py</code>	Resuelve las tres cosas que rompen cualquier lector de XML normal ante un export de DF: está en CP437 y no en UTF-8; lleva dentro bytes de control que en realidad son símbolos (el * de los objetos de calidad es uno de ellos); y pesa demasiado para cargarlo entero. Entrega bytes limpios en streaming.
<code>parser/legends.py</code>	Todo lo que se sabe sobre la <i>forma</i> del XML vive aquí y solo aquí. El resto del programa consulta la base de datos y no necesita saber qué etiquetas existen.
<code>parser/discover.py</code>	Empareja cada <code>legends.xml</code> con su <code>legends_plus.xml</code> . Los sueltos se avisan, no se ignoran en silencio.
<code>parser/organizer.py</code>	DF nombra los exports con el nombre de la carpeta de la partida (<code>region1</code>), que no dice qué mundo es. Este módulo lee el nombre real de los primeros bytes del fichero y ordena la carpeta por mundo y fecha.
<code>parser/importer.py</code>	Del XML a SQLite. Recorre el principal y luego el <code>_plus</code> , fusionando por identificador, y al terminar deduce lo que el XML no dice: la jerarquía de entidades y quién manda en cada sitio cada año.
<code>parser/inspeccion.py</code>	Los informes de «¿Algo no cuadra?». Explicado en el apartado 5.

Lo que se deduce de los datos

FICHERO	QUÉ HACE
<code>model/terreno.py</code>	DF no da un mapa. Da la lista de casillas que ocupa cada región y, aparte, cómo se llama y de qué tipo es. Cruzando las dos cosas sale una rejilla que cubre el mundo entero. Está comprobado que no deja huecos , por eso la costa sale exacta.
<code>model/entities.py</code>	Sube por la cadena de entidades hasta la civilización raíz. Sin esto no se sabe de quién es cada sitio.
<code>model/ownership.py</code>	Reconstruye la propiedad de cada sitio año a año recorriendo los sucesos. Guarda el histórico completo, no solo el estado final: es lo que alimenta el deslizador de año del mapa.
<code>model/fortress.py</code>	Deduce cuál es <i>tu</i> fortaleza (DF no lo marca en ninguna parte) y, si hay duda, te ofrece elegir una vez. Tu elección manda siempre sobre la deducción.
<code>model/diccionario.py</code>	21 tablas y 557 términos de DF a castellano: tipos de sitio, razas, castas, causas de muerte, cargos, vínculos, oficios, habilidades, esferas, objetivos, secretos, biomas.
<code>model/narrador.py</code>	155 plantillas que convierten un suceso en una frase. Es el fichero más grande del proyecto (1.181 líneas) y el que más cambia la sensación de usar el programa.

El servidor y la interfaz

FICHERO	QUÉ HACE
<code>config.py</code>	Todo lo que depende de tu ordenador: dónde está tu carpeta personal, dónde está el <code>.env</code> , qué puerto usar. El resto del código no necesita saberlo.
<code>main.py</code>	El servidor. También hace, al arrancar, las migraciones que hagan falta sin que tengas que enterarte.
<code>api/</code>	31 endpoints repartidos en seis ficheros por tema: mapa y sitios, figuras, fortaleza, mundos, conexión con el juego y crónicas.
<code>ai/context.py</code>	Elige qué hechos se le mandan a la IA y los compacta. Solo datos que están de verdad en la base de datos.
<code>ai/chronicler.py</code>	La única puerta que llama a la API de Anthropic. Mira primero si la crónica ya está guardada.
<code>web/js/atlas.js</code>	El mapa de pergamino, dibujado en canvas. El fondo se pinta una vez en un lienzo aparte y se copia en cada fotograma: por eso el deslizador de año va suelto.

4 Las decisiones que sostienen el resto

La carpeta personal, fuera del programa

Tus exports, tu base de datos, tus crónicas y tu clave viven en `Documentos/ProLegends`, no dentro de la carpeta del programa. Así, cuando bajas una versión nueva, puedes borrar la carpeta vieja entera sin perder nada. Antes había que copiar cosas a mano, y bastaba con despistarse una vez para perder crónicas que habían costado dinero.

Una sola transacción al importar

Un export de 50 MB tardaba **32,2 segundos**. Metiendo toda la carga en una única transacción de SQLite bajó a **9,0 segundos**. También se probó quitar los índices durante la carga —lo que suele recomendarse— y salió *peor*, porque la fase que deduce jerarquías y propiedad los necesita. Queda anotado para no repetir el experimento.

Guardar el registro entero, no solo lo que se usa

Cada tabla guarda además el registro completo en JSON, y hay un cajón de sastre para todo lo que no tiene tabla propia. Eso significa que una vista nueva no obliga a volver a importar los 50 MB: el dato ya está guardado. Es la razón de que la versión 1.5.1 pudiera añadir un informe de tipos de suceso sin tocar el importador.

Los colores no están escritos en el código

Cada raza ocupa una franja del círculo cromático y las civilizaciones de una misma raza se reparten dentro de su franja. El color sale del propio dato, así que dos mundos distintos tienen paletas distintas y coherentes, y no hay ninguna lista de colores por civilización que mantener.

La interfaz no se queda vieja

Como la dirección es siempre `127.0.0.1`, el navegador se quedaba con la interfaz de la versión anterior al actualizar. Ahora el servidor manda una cabecera que obliga a preguntar si ha cambiado, y hay un aviso si el programa y la interfaz no coinciden de versión. Esto se descubrió porque te pasó a ti.

SOBRE TU CLAVE DE LA API

El `.env` está en `.gitignore` desde el primer commit y se ha comprobado que nunca ha entrado en el repositorio. La clave no aparece en ningún informe, ni en la interfaz, ni en los mensajes de error: lo que se enseña es la ruta del fichero, su tamaño y su fecha, que es lo que hace falta para diagnosticar problemas. Por terminal se muestra recortada.

Recordatorio que sigue en pie: la clave que se vio en aquella captura **hay que darla por comprometida**. Si no la revocaste, revócala.

5 Los ríos, y el método que salió de ahí

Merece un apartado porque es el error más caro del proyecto y lo que aprendimos vale para todo lo que venga.

El mapa dibujaba los ríos como un abanico de rayas cruzando el mundo entero. Diagnosticué que las coordenadas venían desordenadas y publiqué un arreglo. No era eso. Volvió a fallar. Tú lo dijiste claro:

«creo que tienes un mal entendido con lo ríos y deberías de comprobar el xml pertinente para arreglarlo»

Tenías razón. Al mirar el fichero de verdad, el recorrido de un río resultó ser **cinco números por punto** —posición, caudal, salida y altura— y yo los estaba leyendo de dos en dos. Cada punto salía de un sitio distinto del mapa: de ahí el abanico. El diagnóstico falso se corrigió en público, en el CHANGELOG.

LA NORMA QUE SALIÓ DE AHÍ

Cuando algo no cuadra, se mira tu dato antes de tocar código. Para eso está el botón «¿Algo no cuadra?»: vuelca la geografía de tu mundo, los tipos de suceso con su recuento, los vínculos y las razas. Si hace falta un dato nuevo para diagnosticar algo, se añade al informe y se te pide; no se adivina.

Esa norma es exactamente la que resolvió el problema de la 1.5.1: los sucesos que no se entendían no eran un fallo del narrador, sino que **cada mundo escribe los nombres a su manera** —`hf died`, `hist figure died`, `change_hf_state`— y el programa solo reconocía una forma.

6 Estado de salud

Lo que está comprobado

La autocomprobación (`tools/autocomprobacion.py`) tiene **19 secciones** y sale entera en verde. No es una lista de comprobaciones de juguete: genera mundos falsos con la misma forma que los de verdad, los importa, dibuja el mapa, narra los sucesos, levanta un servidor real y comprueba hasta las cabeceras del navegador.

SE COMPRUEBA	CÓMO
Que el XML se lee entero, con sus símbolos raros	Mundos generados con * dentro de los nombres
Que el mapa se puede dibujar solo con los datos	Las regiones cubren el 100 % de la rejilla, sin huecos
Que ningún río salta de una punta a otra del mapa	Se mide cada tramo: máximo un paso de casilla
Que sin el <code>_plus</code> se dice «no hay mapa»	En vez de inventarse una geografía
Que las 155 plantillas aguantan un suceso vacío	Se les pasa uno sin datos y ninguna revienta
Que las tres formas de escribir un suceso llevan a la misma frase	Sección 19, añadida en la 1.5.1
Que la versión coincide en los tres sitios donde se escribe	Si no, falla
Que el navegador no se queda con la interfaz vieja	Se levanta un servidor y se mira la cabecera

Rendimiento medido

OPERACIÓN	TIEMPO
Importar un export de 50 MB	9,0 s
Cargar el terreno de un mundo pequeño (48×48)	5 ms
Cargar el terreno de un mundo grande (257×257)	85 ms
Mover el deslizador de año	0,3-3 ms por fotograma

Lo que no está comprobado, dicho claro

- **ABIERTO** **Las relaciones que salen todas como «hijo o hija».** El código que las lee está bien y en los mundos de prueba salen cónyuge, padre, hijo, deidad y amistad correctamente. Hace falta el dato de tu mundo para saber qué pasa. *Está esperando a que pegues el informe.*
- **ABIERTO** **Los años negativos.** En la crónica que me pasaste salía «nació en el año -51». Puede ser real —DF usa años negativos para lo anterior al inicio de la historia— o puede ser un fallo de lectura. No se ha verificado.
- **SIN CUBRIR** **La carpeta `tests/` está vacía.** Es a propósito: la red de seguridad es la autocomprobación, que prueba el programa entero en vez de piezas sueltas. Pero conviene saberlo.
- **SIN CUBRIR** **Las crónicas de IA no se prueban automáticamente,** porque cada prueba costaría dinero. Se prueban a mano cuando se tocan.

7 Las reglas, y cómo van

REGLA QUE PUSISTE	ESTADO	CÓMO SE CUMPLE
La clave de la API no se sube nunca	SE CUMPLE	En <code>.gitignore</code> desde el primer commit; nunca ha entrado en el repositorio
El XML no se carga en el navegador	SE CUMPLE	Se procesa en Python en streaming; al navegador van JSON pequeños
Sin frameworks con compilación ni Node	SE CUMPLE	HTML, CSS y JavaScript a pelo; cuatro dependencias de Python
Nada específico de un mundo en el código	SE CUMPLE	Hasta los colores se derivan del propio export
No inventar campos	SE CUMPLE	Lo que no está en el diccionario se enseña tal cual; sin coordenadas no se dibuja costa
No llamar a la API sin pedirlo	SE CUMPLE	Solo un endpoint llama, y primero mira si ya está guardada
Trabajar siempre en <code>main</code>	SE CUMPLE	34 commits, ninguna rama, ningún pull request

8 Por dónde sigue

El plan completo está en `ROADMAP.md`. En orden:

VERSIÓN	QUÉ TRAE
1.6.0 Interfaz	Buscador global, historial de navegación, direcciones enlazables, tablas ordenables y un sistema visual coherente. Aquí es donde entra el selector castellano / inglés que preguntaste. Es la parte más de gusto: se preparan alternativas y eliges.
1.7.0 Mapa	La segunda mitad del mapa: más detalle de terreno, capas nuevas y los remates que lo convierten en un atlas.
1.8.0 Crónicas	Que las crónicas se dejen leer: mejor selección de hechos y mejor redacción, aprovechando el mismo diccionario de la 1.5.
1.9.0 Sucesos	Los capítulos que agrupan sucesos —guerras, batallas, expediciones— contados como capítulos y no como una lista de líneas sueltas.
2.0.0 Récords	Mayores asesinos, batallas más sangrientas, reinados más largos, y árboles genealógicos dibujados en canvas.

Lo que hay que hacer antes de nada

1. **Pegar el informe de tu mundo.** Mapa → «¿Algo no cuadra?» → copiar. Con eso se cierran las dos dudas abiertas: las relaciones y si tu export deja algún tipo de suceso sin frase propia.
2. **Decidir si la 1.6.0 empieza ya** o si antes se dedica una versión corta a rematar lo que salga del informe.

NUEVO EN ESTE REPASO

Se ha añadido un fichero `CLAUDE.md` en la raíz con las reglas de seguridad, el contexto del proyecto, el mapa del código, las trampas conocidas del entorno y la lista de cosas que no se tocan sin pensarlo mucho. Sirve para que cualquier sesión de trabajo futura —mía o de otro— arranque sabiendo lo que ya sabemos, en vez de volver a descubrirlo.